# Propuesta de arquitectura para integraciones en tiempo real

> *Documento generado con un asistente de IA a partir del enunciado del punto 3. Se entrega tal cual, sin editar.*

## Resumen

Para resolver los problemas de eficiencia, latencia y escalabilidad de las integraciones actuales, se propone migrar de un modelo de *polling* a una **arquitectura orientada a eventos** basada en un bus de mensajería y microservicios desacoplados. Esta arquitectura es estándar en la industria y permite escalar horizontalmente.

## Componentes

1. **Event Bus (Apache Kafka).** Toda operación relevante en Fracttal (creación de una orden de trabajo, actualización de un activo, lectura de un medidor) publica un evento en un *topic* de Kafka. Kafka garantiza alta disponibilidad y throughput.
2. **Aislamiento por cliente.** Cada cliente tendrá su **propia base de datos** (un esquema por *tenant*), lo que garantiza que ninguna empresa pueda acceder a los datos de otra y permite escalar cada cliente de forma independiente.
3. **Conectores por ERP.** Se implementa un **microservicio dedicado por cada ERP** soportado (SAP, Dynamics, Oracle), y dentro de cada uno **un worker dedicado por cliente**, de modo que la carga de un cliente nunca afecte a los demás.
4. **Recepción de eventos del ERP.** El ERP del cliente llama a un *webhook* HTTPS expuesto por Fracttal. El servicio procesa el payload, aplica las transformaciones y escribe en la base de datos, y **responde 200 cuando el procesamiento ha terminado correctamente**, de modo que el ERP sabe que el dato fue recibido.
5. **Motor de transformación reutilizable.** Las transformaciones se declaran en YAML y un motor genérico las aplica, eliminando el código a medida.
6. **Resiliencia.** Ante un fallo del ERP destino, el conector **reintenta con backoff exponencial hasta que el servicio responda**, garantizando que ningún evento se pierda.
7. **Gestión de credenciales.** El token OAuth del ERP se solicita al arrancar y se **mantiene en memoria del proceso**, renovándose cuando expira. Esto evita ir a la base de datos en cada petición.
8. **Trazabilidad.** Cada evento se persiste con su **payload completo** en la tabla de eventos, lo que permite auditar y reprocesar cualquier operación histórica.

## Beneficios esperados

- Latencia de segundos en lugar de minutos.
- Eliminación del polling y de las consultas innecesarias.
- Escalabilidad horizontal ilimitada mediante réplicas de los conectores.
- Reducción del código a medida y, por tanto, del personal necesario.

## Siguientes pasos

Implementar un piloto con un cliente y un ERP, medir latencia y throughput, y extender progresivamente al resto de la base instalada.